mb-free___transport-eager-128

An AgentMPI run analysis

Abstract

This is the floor of the eager column in a ten-cell transport sweep — two modes crossed with five payload sizes, all at $p = 8$ and all agent-free. The program is a ring in which every rank sends one 128-word payload to its successor and then receives from its predecessor, with the transport mode pinned to `Mode.EAGER` rather than left to the runtime's threshold. It completed in 0.02 s over 34 events, carried all eight messages, and admitted 168 tokens into each receiver's context: 0.5% of the 32,768-token context budget and 4.1% of the 4,096-token unexpected-message credit that governs the eager path. Nothing here is surprising, and that is the cell's function — it is the calibration point against which the two eager cells that did not complete, at 8,192 and 32,768 words, have to be read. The thing to carry away is what the 168 represents. Eager transport charges the receiver, in context, the full token cost of whatever is sent to it, and that charge is the resource the rest of the sweep goes on to exhaust.

1 What this run is

property	value
run	`mb-free__transport-eager-128`
experiment	-
campaign label	-
declared world size	8
ranks appearing in the log	8
executors	`none` × 8
events	34
wall time	0.02 s
job outcome	completed

2 Headline measurements

metric	value	meaning
achieved parallelism	0.00×	total busy time over wall time
parallel efficiency	0.0%	achieved parallelism per rank
idle fraction	100.0%	rank-seconds paid for and unused
peak ranks busy	0	of 8 in the world
serial fraction of active time	0.0%	time with exactly one rank busy
load imbalance	0.00×	slowest rank's busy time over the mean
coordination share	0.0%	rank-seconds blocked in collectives, of those available
coordination in flight	0.0%	wall clock with any rank inside a collective
rank reattachments	0	fresh agent processes that took over a rank role
agent calls	0	invocations that reached a model
agent latency p50 / p95	0.00 / 0.00 s	per invocation
messages	8	8 eager, 0 rendezvous
tokens sent	1,344	0 deferred under rendezvous
tokens in / out	0 / 0	prompt and completion
cost	$0.0000	0.0000 $/kTok out

3 What the analysis notices

Nothing anomalous was detected mechanically.

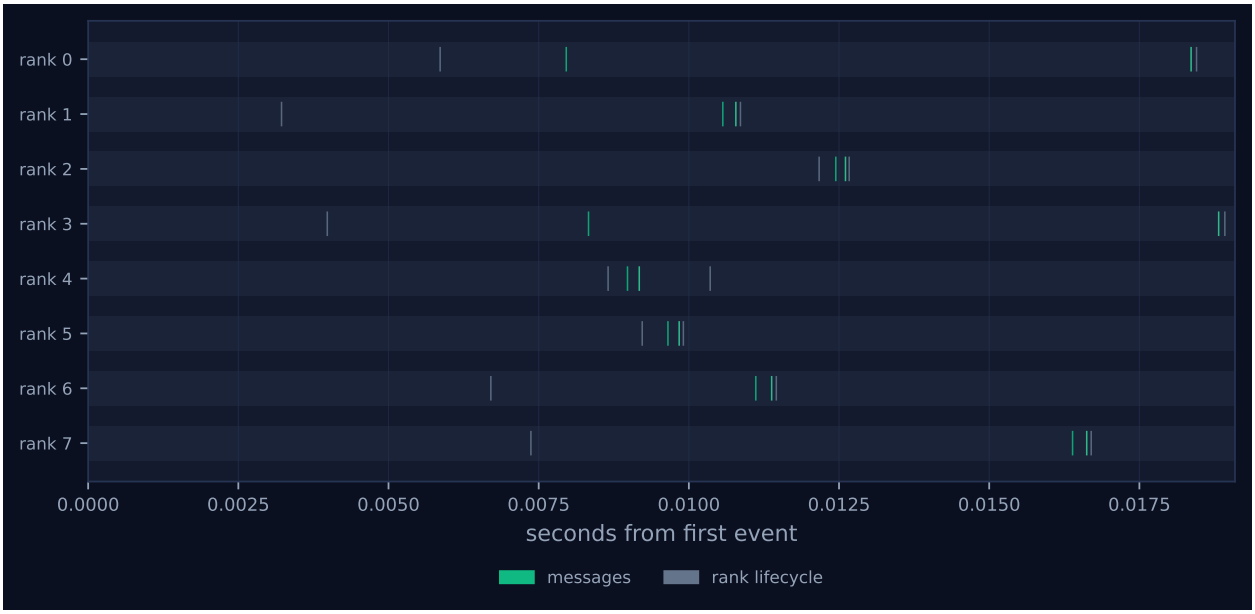

Figure 1: Execution timeline, one lane per rank. Bars are intervals when a rank held a task; ticks are messages; diamonds are window operations, lifecycle transitions, failures, and recovery actions. Drawn in the trace viewer's palette so the same shapes read the same way in both.

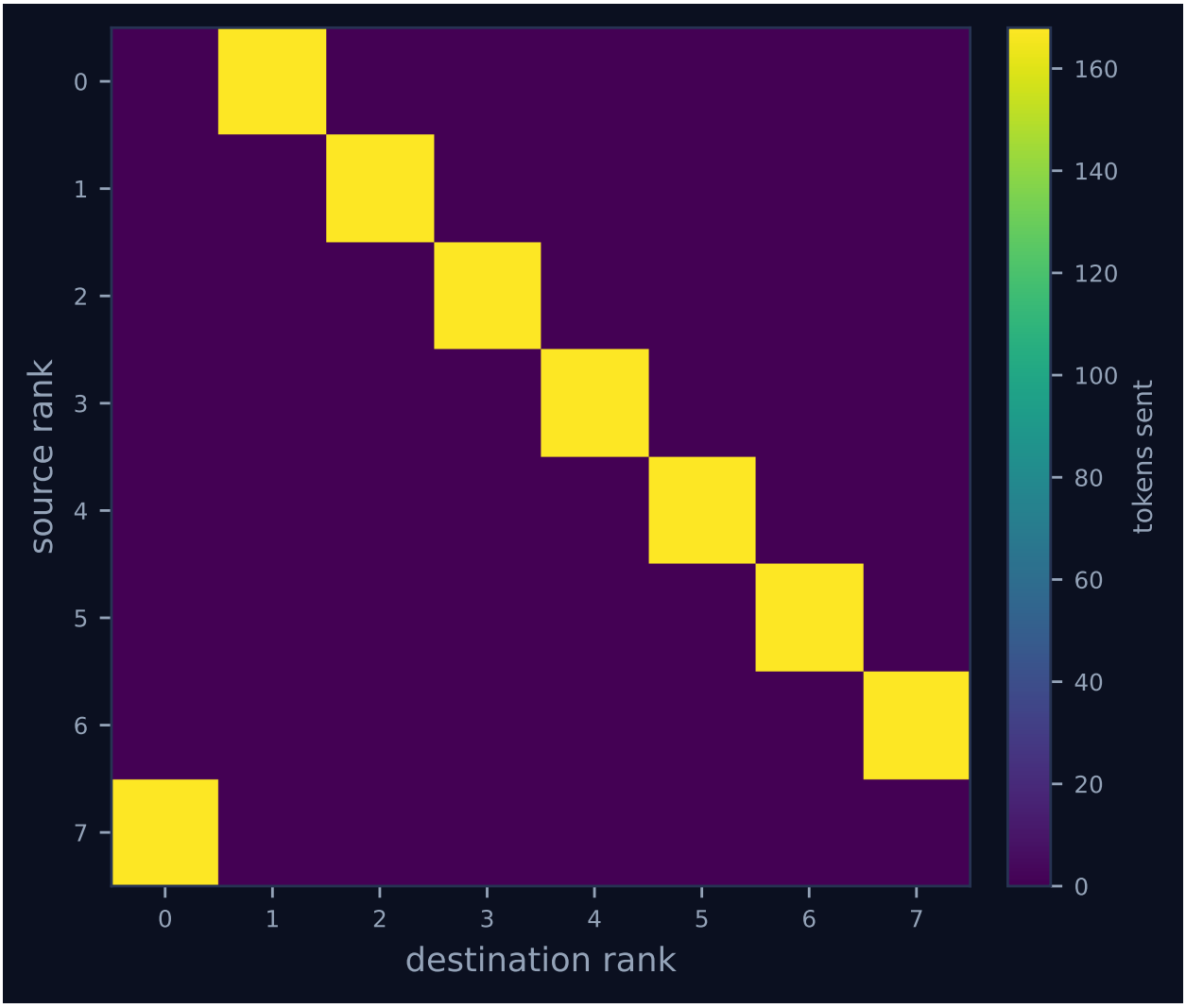

Figure 2: Communication matrix: tokens sent from each rank to each rank. The algorithm's shape is visible here — a tree concentrates traffic on a root, a ring forms a diagonal band, and an unintended all-to-all fills the plane.

4 Per-rank detail

rank	busy (s)	occ. (%)	tasks	calls	sent	recv	tok. sent	ctx (%)	state
0	0.00	0.0	0	0	1	1	168	0.5	finalized
1	0.00	0.0	0	0	1	1	168	0.5	finalized
2	0.00	0.0	0	0	1	1	168	0.5	finalized
3	0.00	0.0	0	0	1	1	168	0.5	finalized
4	0.00	0.0	0	0	1	1	168	0.5	finalized
5	0.00	0.0	0	0	1	1	168	0.5	finalized
6	0.00	0.0	0	0	1	1	168	0.5	finalized
7	0.00	0.0	0	0	1	1	168	0.5	finalized

Per-rank behaviour. Occupancy is busy time over the rank's own lifetime, so a rank that joined late is not penalised for the time before it existed.

5 Collectives, against the cost model

This run invoked no collectives.

6 Reading the timeline

Figure 1 shows eight lanes carrying four instantaneous marks each and no bars anywhere. The absence of extent is the first thing to explain: the executor is `none` on all eight ranks and there were zero agent calls, so no `task.*` event was ever emitted and there is nothing for a bar to span. The headline table's "achieved parallelism 0.00×" and "idle fraction 100.0%" therefore say that no rank ever held a task, not that ranks sat waiting on each other; they are properties of an agent-free benchmark and carry no information about the protocol. What the lanes do carry is two grey lifecycle diamonds per rank (`rank.init`, `rank.finalize`) and two green message ticks (`msg.send`, `msg.recv`). Ranks arrive over a 9 ms window, from rank 1 at 3 ms to rank 2 at 12 ms, which is roughly half the 19 ms total span and is thread start-up against a single SQLite writer rather than anything the ring did.

The structural fact visible in the lanes is that no rank has a gap between its send and its receive. The largest message latency in the log is 10 ms and the median is 4 ms, and every rank finalises within 1 ms of its own receive. Neither figure is protocol delay: `sent_at` is stamped just before the sender opens its write transaction, so a latency measures how long the row took to commit and be claimed while eight ranks contend for one SQLite writer. Nobody blocked on a peer. That matters because the ring is deliberately the shape MPI calls an unsafe program: every rank sends before it receives, so under bounded buffering it can deadlock. At 168 tokens against 4,096 tokens of credit there is nothing to deadlock on, and the send returns as soon as the row is in the fabric. The ring's topology is not legible in the timeline at all, because ticks have no direction; Figure 2 is where it appears, as a single off-diagonal band of 168 tokens from rank r to rank $(r{+}1) \bmod 8$, one message per edge and eight edges. That band is exactly what the failed eager cells at 8,192 and 32,768 words do not have: their communication matrix is empty and `analyze_run.py` does not draw one.

7 Interpretation

Most of this run is true by construction. The world size, the single send and single receive per rank, the ring's successor mapping and the choice of eager are all harness decisions, and one of them deserves naming because it is easy to miss: every `rank.init` event records `eager_limit: 1000000000`. The runtime's automatic threshold, which would otherwise pick rendezvous above `DEFAULT_EAGER_LIMIT` = 2048 tokens, is switched off. That is what makes transport mode an independent variable in this sweep rather than a function of payload size, and it is why the two large eager cells were allowed to attempt something the runtime would ordinarily have converted to rendezvous on their behalf.

What emerged is the price. The payload is 128 repetitions of `"word "`, 640 characters, and AgentMPI's default offline estimator prices it at 168 tokens — the maximum of $640/3.8 = 168.4$ and $128 \times 0.82 = 105$. All eight `msg.recv` events report `admitted: true` with `admitted_tokens: 168`, and all eight ranks finalise with a context high water of 168 over one item and zero rejections. The mechanism behind that uniformity is the receive default: the harness calls `recv` without an `admit` argument, and `Communicator._deliver` resolves `admit = None` to `mode != rendezvous`. The sender's choice of eager became the receiver's decision to spend context, which is precisely the semantics `constants.Mode` claims for the distinction. Note also that the generated table's "0 deferred under rendezvous" is not a vacuous entry. `RankCost.tokens_deferred` accumulated on both rendezvous sends *and* non-admitted receives until commit `0fea51d2` split it from `tokens_unadmitted`, and this cell reads zero under either definition specifically because it admits. The eager rows of the published transport table are sound for that reason; the rendezvous rows of the same table are not, for reasons the rendezvous cells of this sweep make explicit.

Placed against its siblings, 168 tokens is 4.1% of the per-destination credit and the charge is linear in payload with no fixed overhead: 512 and 2,048 words cost 674 and 2,695 tokens, which is 4.01× and 16.04× for 4× and 16× the words. Extending the same line, the 8,192-word payload prices at 10,779 tokens, 2.6× the credit, and `Communicator._await_eager_credit` refuses it on a precondition before writing anything to the fabric at all. The eager ceiling therefore lies between the 2,048-word cell and the 8,192-word cell; under this estimator the exact crossing is 3,113 repetitions of `"word "`. Nothing about the present cell is near it, and the sweep does not argue that it should be — at half a percent of a receiver's budget, eager is the right mode and rendezvous would buy only the loss of the content. The generated finding list is empty, and that is correct: there is no anomaly here to explain away.

8 Threats to this reading

Four things could make this reading wrong. First, the executor is `none` and there were no agent calls, so every context figure in this document is an accounting entry rather than a measurement — `ContextAccount` recorded that 168 tokens were admitted, and no model ever read them. The run establishes what the protocol charges, not what an agent does with the charge. Second, the token count is an estimate: the provenance block records `tokenizer_exact: false`, and 168 is $\lceil 640/3.8 \rceil$ from a character-based heuristic. A real BPE tokeniser would price 128 repetitions of a single common word nearer 128 tokens, which would shift every threshold in this sweep by roughly 30% — the ceiling's location in words is an artefact of the estimator even though its existence is not, and the payload, being one word repeated, is unrepresentative of prose in exactly the way that matters to a tokeniser. Third, the wall time is one sample and is not a measurement of anything

protocol-related: 19 ms total, of which 9 ms is rank start-up spread, against SQLite writes on one file. No cross-cell wall-time comparison in this sweep survives that. Fourth, and most limiting for the sweep's larger argument, a ring gives each receiver exactly one inbound message, so the accumulation that the eager credit mechanism exists to bound is never exercised: there are no `transport.credit_stall` events in any of the ten runs. What this cell tests is a per-message precondition, not back-pressure. A fan-in of two 2,048-word senders into one rank would exceed the same 4,096-token credit by accumulation and would exercise the blocking path instead, and that experiment does not exist here.